



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Achmad Faiz Yudha Ramadhan

NIM : 2428240113

Kelas : SI5C

**PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027**

Identitas Praktikum

Nama Mahasiswa	Achmad Faiz Yudha Ramadhan
NPM	2428240113
Kelas	SI5C
Pertemuan Ke-	4
Topik / Judul Praktikum	Tugas 1 RESTful API
Tanggal Pelaksanaan	24 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom

I. Tujuan Praktikum

Praktikum ini bertujuan untuk memahami dan menerapkan konsep RESTful API menggunakan Node.js dan Express.js dalam pengelolaan resource film bioskop.

1. Memahami konsep RESTful API dan penggunaan method HTTP untuk mengelola resource.

2. Membuat RESTful API menggunakan Express.js dengan resource /movies.

3. Menerapkan operasi CRUD yang terdiri dari GET, POST, PUT, dan DELETE pada data film.

4. Menerapkan validasi input, query string untuk filter genre, serta response JSON dan status code yang sesuai.

5. Melakukan pengujian endpoint menggunakan Postman serta melakukan deployment source code ke GitHub dan Vercel.

II. Dasar Teori

RESTful API (Representational State Transfer Application Programming Interface) merupakan layanan yang mengelola resource melalui URL dan method HTTP. Pada praktikum ini resource yang dikelola adalah film pada endpoint /movies. RESTful API tidak menampilkan halaman HTML, tetapi mengembalikan data dalam format JSON sehingga dapat digunakan oleh client seperti Postman. Operasi terhadap resource dilakukan menggunakan method GET untuk membaca data, POST untuk menambahkan data, PUT untuk mengganti data secara keseluruhan, dan DELETE untuk menghapus data.

Express.js merupakan framework backend untuk Node.js yang digunakan untuk membuat server dan routing HTTP. Middleware `express.json()` digunakan untuk membaca request body dalam format JSON. Data pada praktikum disimpan dalam array di memory sesuai ketentuan tugas dan ID dibuat otomatis menggunakan `nextId`. API juga menggunakan query string melalui parameter genre, contohnya `/movies?genre=Drama`, untuk menyaring data film. Response GET diberikan langsung sebagai objek atau array, sedangkan POST, PUT, DELETE, dan error menggunakan struktur JSON berisi status, message, dan data. Status code yang digunakan adalah 200 untuk request berhasil, 201 untuk pembuatan data baru, 400 untuk validasi yang tidak sesuai, dan 404 ketika data atau endpoint tidak ditemukan.

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime JavaScript
2	Visual Studio Code	Code Editor
3	Express.js	Framework Backend
4	POSTMAN	Pengujian API
5	Github	Repository Source Code
6	Vercel	Deployment Aplikasi

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1. Memastikan Node.js telah terpasang, kemudian membuat folder project baru untuk RESTful API Bioskop dan melakukan inisialisasi project menggunakan perintah `npm init -y`.

2. Menginstall Express.js sebagai dependency dan Nodemon sebagai devDependency, kemudian mengatur package.json agar menggunakan app.js sebagai entry point serta menyediakan script `npm start` dan `npm run dev`.

3. Membuat file app.js, menginisialisasi Express.js, menggunakan `app.use(express.json())`, serta menyiapkan array minimal tiga data film dan variabel `nextId` untuk pembuatan ID otomatis.

4. Membuat endpoint `/movies` yang terdiri dari GET untuk mengambil data, POST untuk menambah data, PUT untuk mengubah seluruh data, DELETE untuk menghapus data, GET `/movies/:id` untuk mengambil satu data, dan GET `/movies?genre=Drama` untuk filter berdasarkan genre. Validasi field wajib dan penanganan error 400 serta 404 diterapkan pada route yang sesuai.

5. Menjalankan server secara lokal dan melakukan pengujian endpoint menggunakan Postman, meliputi skenario berhasil dan skenario gagal seperti data tidak ditemukan serta field wajib kosong. Setiap pengujian dicatat dan hasilnya digunakan sebagai bukti praktikum.

6. Membuat repository GitHub public, melakukan commit secara bertahap, mengunggah source code, menyesuaikan app.js untuk lingkungan serverless, melakukan deployment ke Vercel, dan menguji kembali endpoint melalui URL deployment.

V. Hasil dan Pembahasan

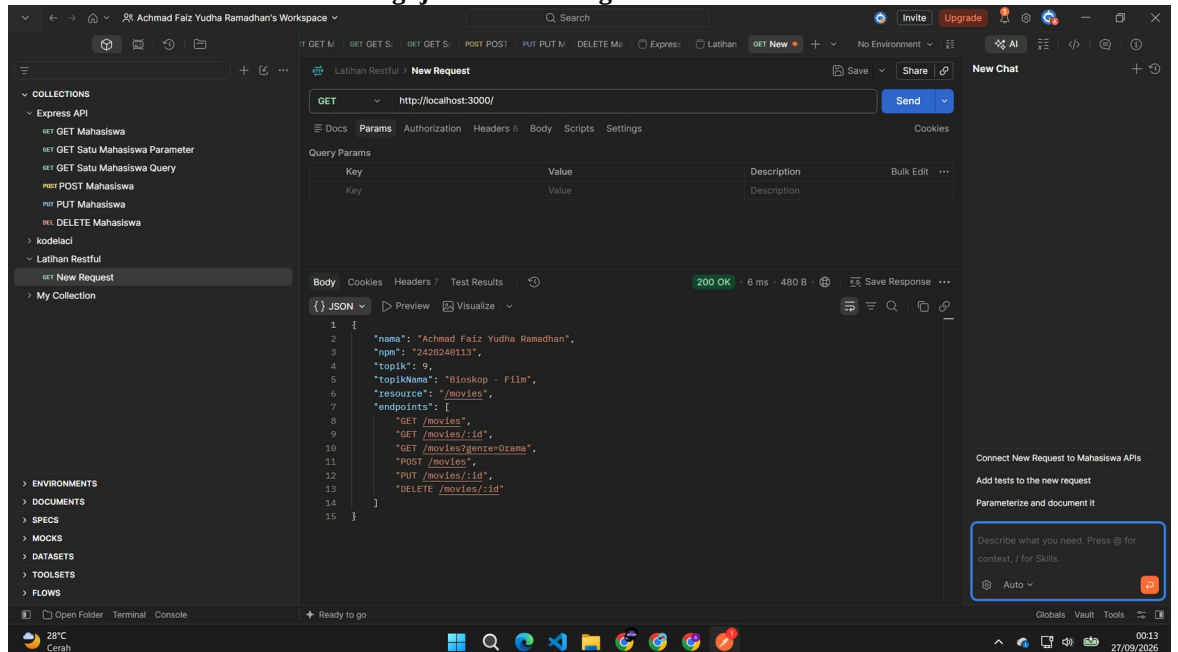
Hasil praktikum menunjukkan bahwa RESTful API untuk resource film bioskop pada endpoint /movies telah dapat menjalankan operasi CRUD, filter berdasarkan genre, validasi field wajib, serta menghasilkan response JSON dengan status code sesuai ketentuan. Pengujian dilakukan menggunakan Postman pada server lokal <http://localhost:3000>. Tabel berikut merangkum hasil pengujian yang digunakan sebagai bukti pelaksanaan praktikum.

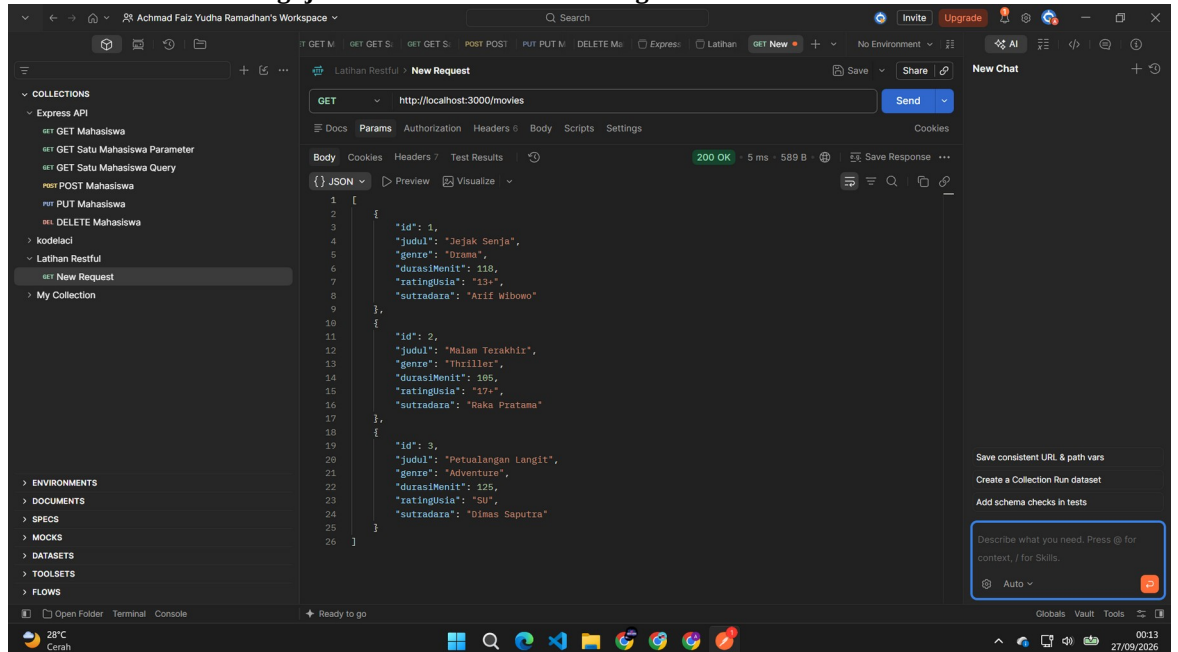
Tabel 1. Hasil Pengujian RESTful API

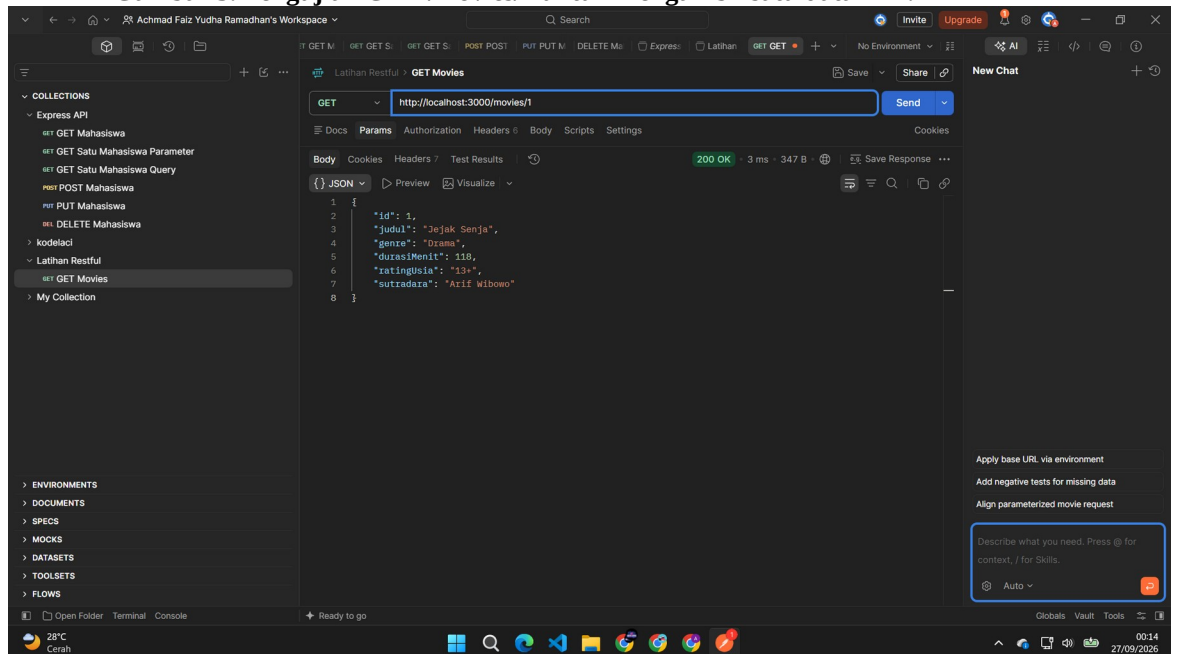
No	Method	Endpoint	Data request	Status diharapkan	Status hasil
1	GET	/movies	—	200	200
2	GET	/movies/1	—	200	200
3	GET	/movies/99	—	404	404
4	GET	/movies?genre=Drama	—	200	200
5	POST	/movies	body lengkap	201	201
6	POST	/movies	field wajib kosong	400	400
7	PUT	/movies/1	body lengkap	200	200
8	PUT	/movies/99	body lengkap	404	404
9	DELETE	/movies/4	—	200	200
10	DELETE	/movies/99	—	404	404

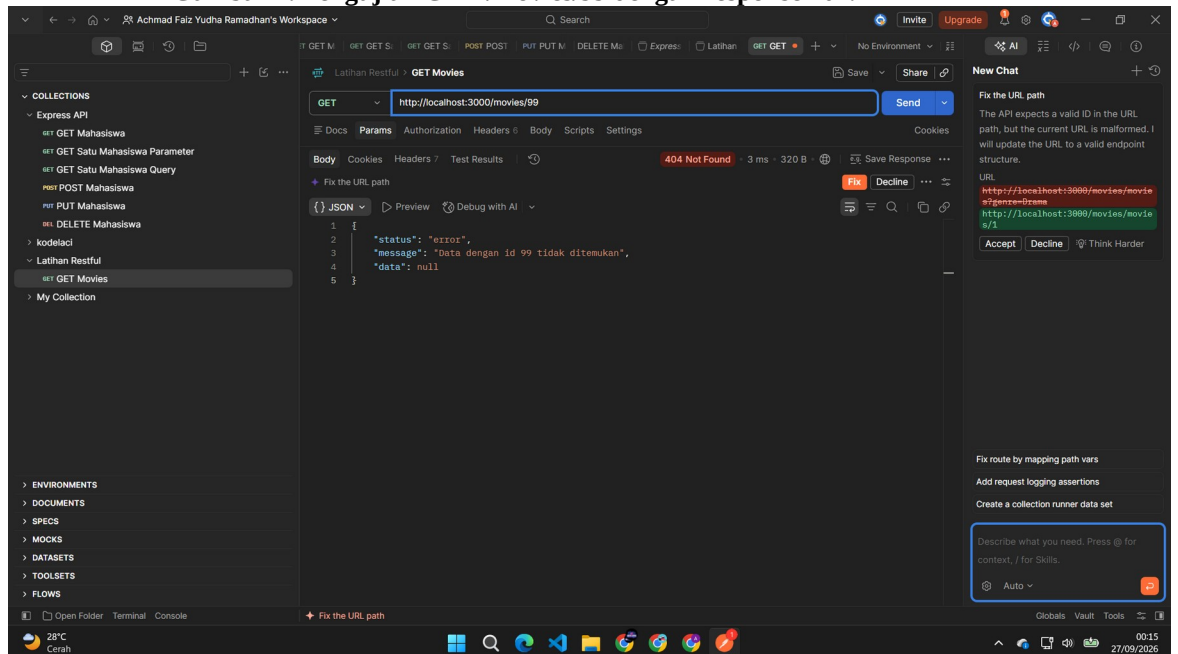
Catatan: Pada bukti pengujian DELETE digunakan ID 4 karena ID tersebut merupakan data film yang ditambahkan pada pengujian POST. Endpoint tetap menggunakan pola RESTful DELETE /movies/:id.

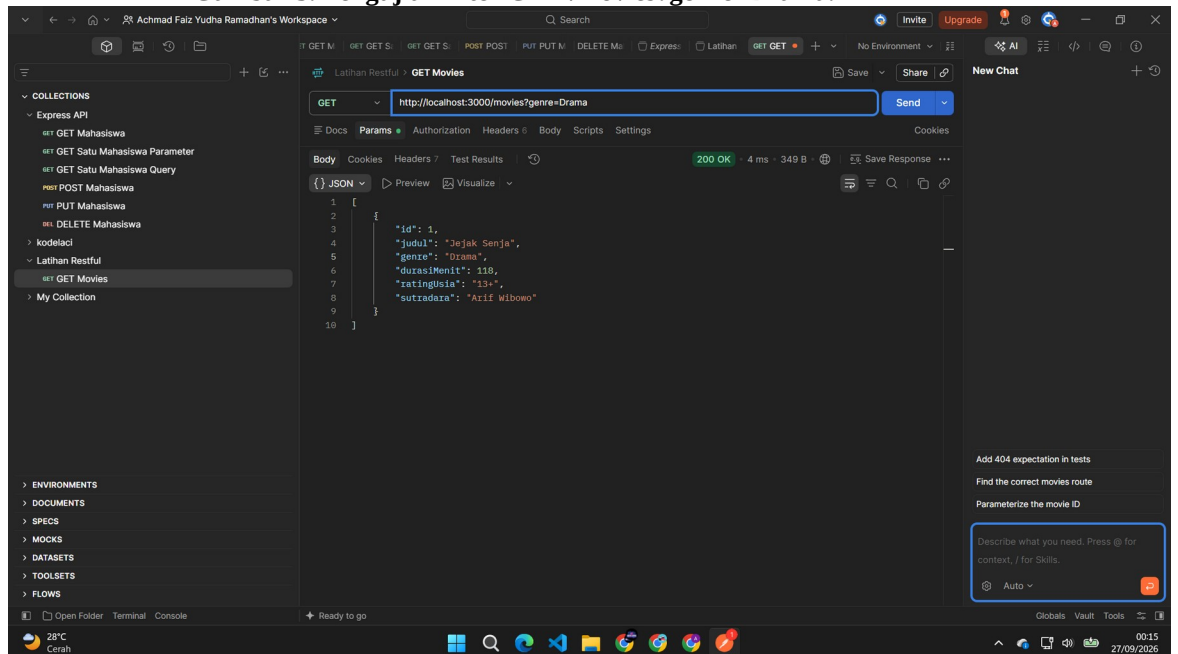
Gambar 1. Pengujian GET / sebagai informasi API.

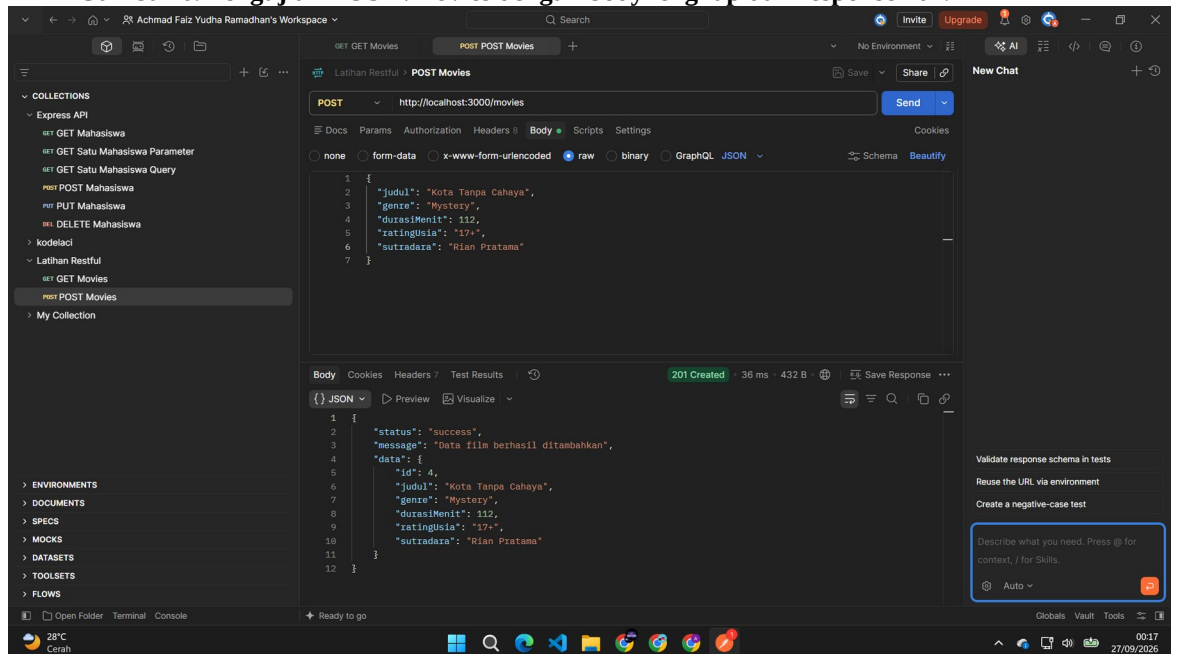


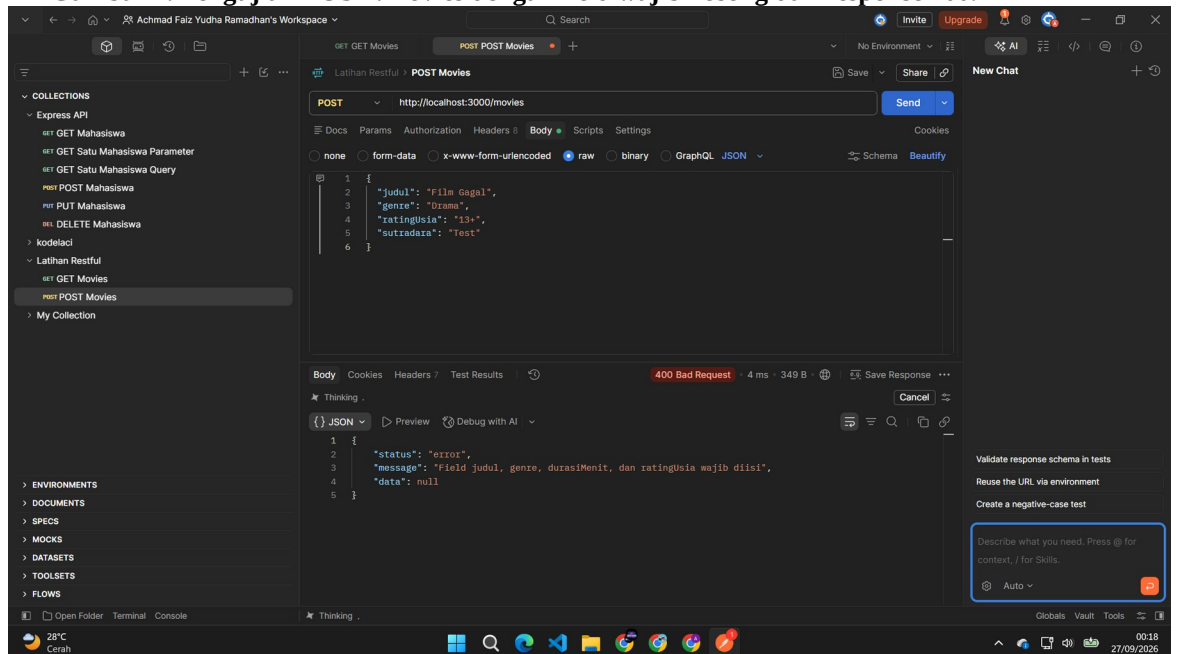
Gambar 2. Pengujian GET /movies untuk mengambil seluruh data film.

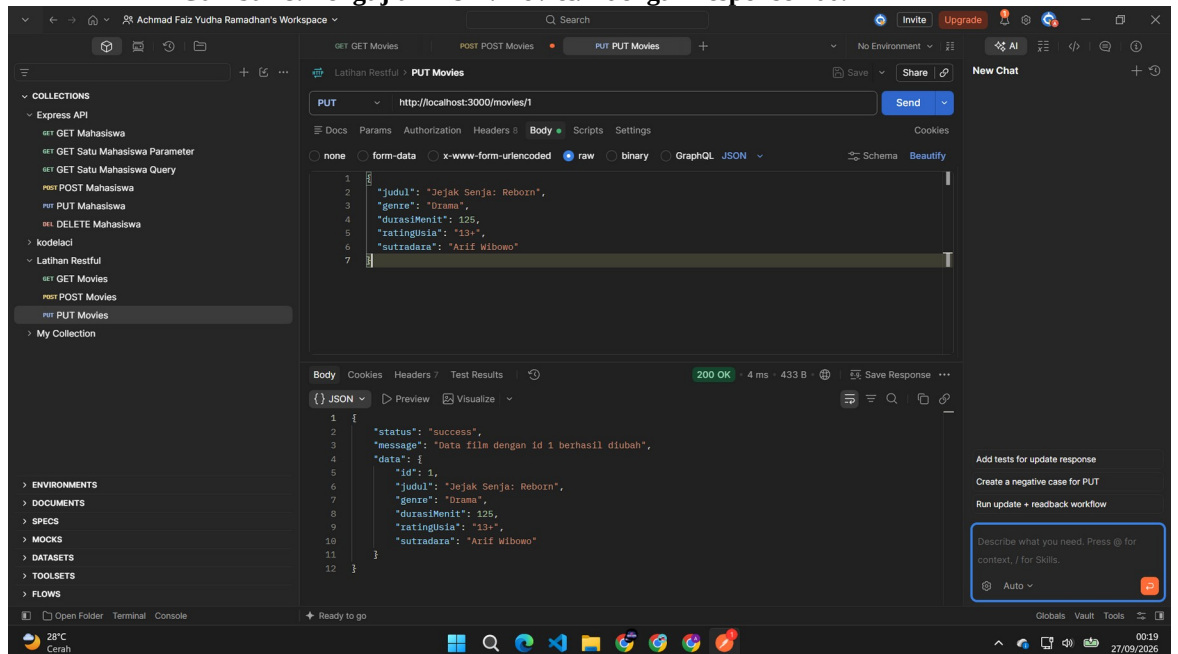
Gambar 3. Pengujian GET /movies/1 untuk mengambil satu data film.

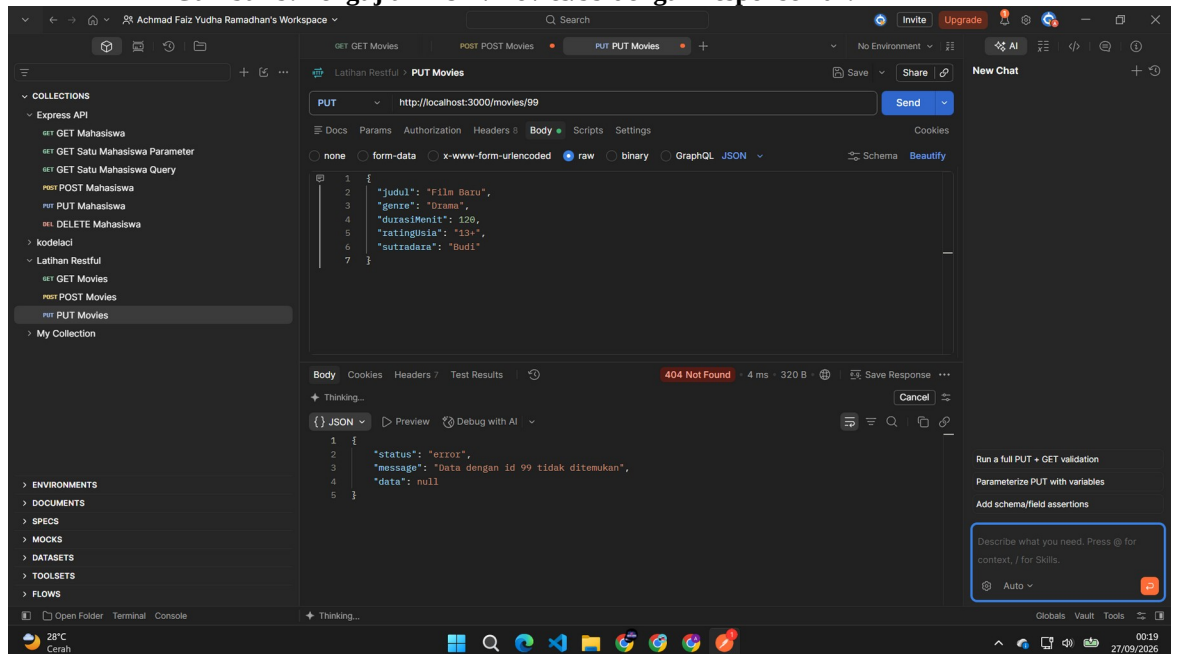
Gambar 4. Pengujian GET /movies/99 dengan response 404.

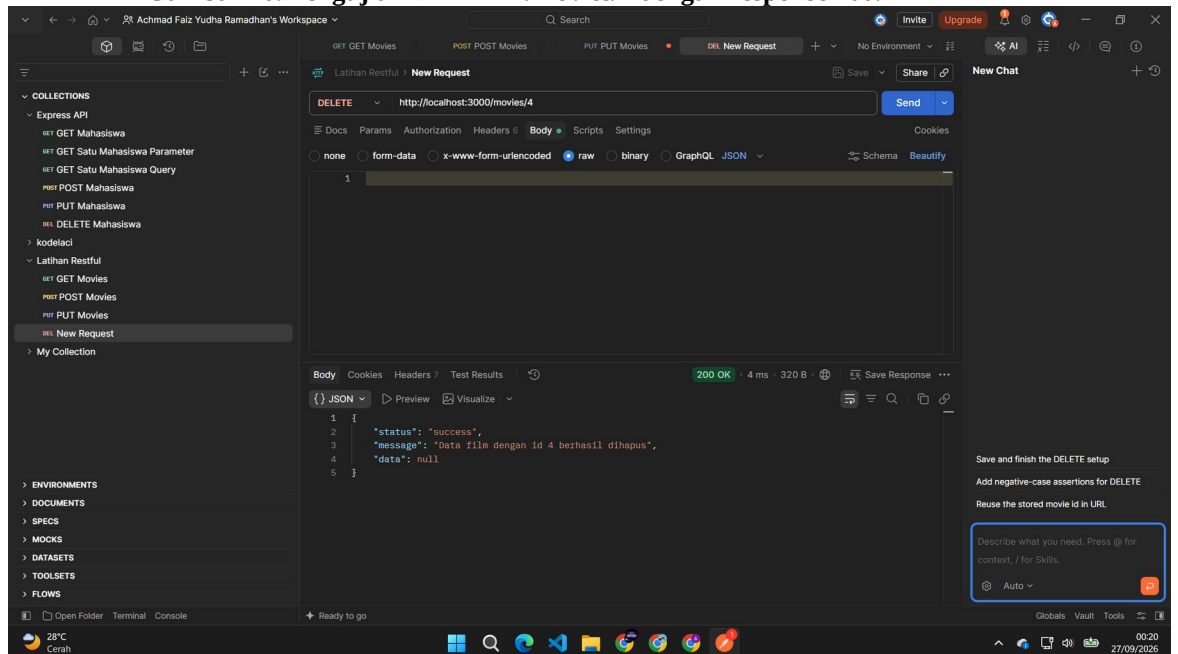
Gambar 5. Pengujian filter GET /movies?genre=Drama.

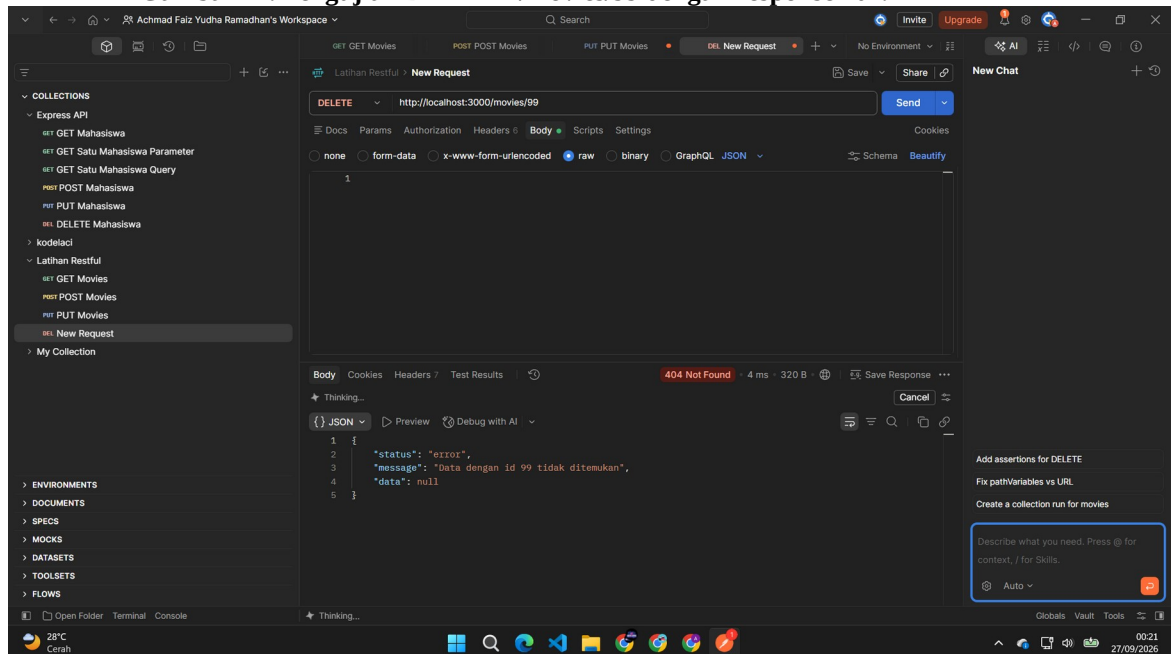
Gambar 6. Pengujian POST /movies dengan body lengkap dan response 201.

Gambar 7. Pengujian POST /movies dengan field wajib kosong dan response 400.

Gambar 8. Pengujian PUT /movies/1 dengan response 200.

Gambar 9. Pengujian PUT /movies/99 dengan response 404.

Gambar 10. Pengujian DELETE /movies/4 dengan response 200.

Gambar 11. Pengujian DELETE /movies/99 dengan response 404.

Pembahasan Hasil Pengujian:

Pengujian GET /movies digunakan untuk mengambil seluruh data film dan menghasilkan array data dengan status code 200. GET /movies/1 digunakan untuk mengambil satu data berdasarkan ID dan menghasilkan objek film dengan status code 200. Ketika ID 99 digunakan melalui GET /movies/99 dan data tidak tersedia, API mengembalikan response error dengan status code 404.

Pengujian GET /movies?genre=Drama digunakan untuk menerapkan filter menggunakan query string genre. Jika data sesuai ditemukan, API mengembalikan array hasil filter dengan status code 200. Pengujian POST /movies dengan body lengkap digunakan untuk menambahkan film baru, ID dibuat otomatis, dan response menggunakan status code 201.

Pengujian POST /movies dengan field wajib yang kosong digunakan untuk memastikan validasi berjalan. Request ditolak dan menghasilkan status code 400. Pengujian PUT /movies/1 digunakan untuk mengganti seluruh data film berdasarkan ID dan menghasilkan status code 200, sedangkan PUT /movies/99 menghasilkan status code 404 karena data tidak ditemukan.

Pengujian DELETE /movies/4 digunakan untuk menghapus film yang sebelumnya ditambahkan melalui pengujian POST. Request berhasil dan menghasilkan status code 200 dengan data bernilai null. Pengujian DELETE /movies/99 menghasilkan status code 404 karena data tidak ditemukan. Secara keseluruhan, hasil pengujian menunjukkan bahwa endpoint mengikuti pola RESTful, menggunakan route parameter hanya untuk ID, query string untuk filter, serta format response JSON sesuai ketentuan tugas.

VI. Kendala dan Solusi

Selama pengembangan tidak terdapat kendala yang menghambat penyelesaian API. Beberapa penyesuaian dilakukan untuk memastikan aplikasi dapat dijalankan pada lingkungan lokal dan kompatibel dengan deployment Vercel.

No.	Kendala	Solusi
1	Penyesuaian server untuk deployment Vercel.	Menggunakan <code>process.env.PORT</code> , menjalankan <code>app.listen</code> hanya pada lingkungan non-production, dan mengekspor app dengan <code>module.exports = app</code> .
2	Data disimpan dalam array di memory sehingga dapat kembali ke data awal pada lingkungan serverless.	Mengikuti ketentuan tugas yang tidak menggunakan database dan memfokuskan pengujian pada kebenaran response setiap endpoint.

VII. Kesimpulan

Praktikum berhasil menghasilkan RESTful API sederhana untuk resource film bioskop menggunakan Node.js dan Express.js. API memiliki operasi CRUD pada endpoint `/movies`, filter berdasarkan genre menggunakan query string, validasi field wajib, dan response JSON dengan status code yang sesuai. Pengujian menggunakan Postman dilakukan pada skenario berhasil dan gagal untuk memastikan setiap endpoint dapat menangani request dengan benar. Source code kemudian disimpan pada repository GitHub dan aplikasi dipersiapkan untuk deployment ke Vercel. Dengan demikian, tujuan praktikum untuk memahami dan menerapkan konsep RESTful API menggunakan Express.js telah tercapai.

Daftar Pustaka

Express.js. (2026). *Express - Fast, unopinionated, minimalist web framework for Node.js*. <https://expressjs.com>

Node.js Foundation. (2026). *Node.js Documentation*. <https://nodejs.org>

Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. University of California.

Lampiran

Tautan Repository GitHub:

<https://github.com/FaizZzhue/tugas1-restful-2428240113>

Tautan Deployment (jika ada):

<https://tugas1-restful-2428240113.vercel.app/>